

# Chuleta examen - Entorno Cliente (JavaScript)

Oct 5, 2026 · @miguel

## 1. Ventanas (cae en examen)

Se abre con `window.open` y se guarda la referencia en una variable; con ella la ventana padre controla a la hija.

```
let nuevaVentana = window.open("URL", "nombre", "parametros");  
// URL vacía "" => ventana en blanco: se escribe con  
nuevaVentana.document.write("<h1>Hola</h1>")
```

Parámetros: se separan por comas, sin espacios problemáticos. `0` o `no` = no tiene, `1` o `yes` = sí tiene.

```
"toolbar=0,location=0,menubar=0,status=0,scrollbars=1,resizable=1,width=300,height=200,top=500,left=500"
```

| Parámetro      | Qué controla                                                |
|----------------|-------------------------------------------------------------|
| toolbar        | barra de herramientas                                       |
| location       | barra de direcciones                                        |
| menubar        | barra de menú                                               |
| status         | barra de estado                                             |
| scrollbars     | barras de scroll                                            |
| resizable      | se puede redimensionar                                      |
| width / height | ancho / alto en píxeles                                     |
| top / left     | posición desde el borde superior / izquierdo de la pantalla |

Desde la ventana padre (con la variable `v`):

```

v.document.write("<h1>Hola</h1>");          // escribir en la hija
v.document.body.innerHTML += "<p>texto</p>"; // añadir contenido
v.document.body.style.backgroundColor = "green";
v.moveBy(200, 100);    // mover relativo (px a la derecha, px abajo)
v.moveTo(500, 200);    // mover a posición absoluta
v.resizeBy(100, 100);  // aumentar tamaño
v.resizeTo(400, 200);  // tamaño exacto
v.focus();             // traer al frente (hacerlo tras mover/redimensionar)
v.close();             // cerrar la hija
if (v.closed) { }     // true si está cerrada

```

**Desde la ventana hija** (acceder a la que la abrió):

```

window.opener.document.body.style.backgroundColor = "red";
window.opener.document.getElementById("id").innerHTML = "texto";
window.close();           // cerrarse a sí misma
window.opener.close();    // cerrar la padre

```

**Dos ventanas hijas que se comunican:** se guardan ambas en `var` globales en el padre (`ventana1`, `ventana2`); desde una hija se llega a la otra con

```

window.opener.ventana2.document...

```

**Orden en un botón:** `<button type="button" onclick="funcion()">Texto</button>`; el JS va en un fichero externo `<script src="js/accion.js"></script>` dentro del `<head>`.

El navegador puede bloquear las ventanas emergentes: hay que permitir las.

## 2. Temporizadores

El tiempo va en milisegundos (1 s = 1000). Se pasa el **nombre de la función** sin paréntesis (`f1`, no `f1()`).

```

let t1 = setTimeout(f1, 5000);    // se ejecuta UNA vez a los 5 s
clearTimeout(t1);                 // anula el setTimeout

let t2 = setInterval(f1, 1000);   // se ejecuta CADA 1 s
clearInterval(t2);                // detiene el setInterval

function f1() { /* instrucciones */ }

```

**Redirigir tras 5 s:**

```

setTimeout(irAGoogle, 5000);
function irAGoogle() { window.location.href = "https://www.google.es"; }

```

**Cuenta atrás que cierra una ventana:**

```

let restantes = 10;
let v = window.open("emergente.html", "", "width=400,height=200");
let intervalo = setInterval(contar, 1000);
function contar() {
    restantes--;
    if (restantes <= 0) {
        clearInterval(intervalo);    // siempre parar el intervalo
        v.close();
    } else {
        v.document.body.innerHTML = `<p>Quedan ${restantes} s</p>`;
    }
}

```

**Reloj que se para a los 5 s y cierra a los 10 s:**

```

let seg = 0;
let reloj = setInterval(function () {
    seg++;
    if (seg <= 5) v.document.body.innerHTML = `<p>${new
Date().toLocaleTimeString()}</p>`;
    if (seg == 10) { clearInterval(reloj); v.close(); }
}, 1000);

```

**Alternar colores de fondo hasta pulsar Detener:**

```

let i = setInterval(cambia, 1000);
function cambia() {
    document.body.style.backgroundColor =
        document.body.style.backgroundColor == "orange" ? "blue" : "orange";
}
function detener() { clearInterval(i); }

```

Regla: `setTimeout` es una vez, `setInterval` se repite hasta `clearInterval`; guardar siempre el temporizador en una variable para poder anularlo.

### 3. Date

Los meses empiezan en 0 (enero = 0, diciembre = 11). `getDay()` da 0 = domingo hasta 6 = sábado.

```
let f = new Date(); // ahora
let f2 = new Date(milisegundos);
let f3 = new Date(año, mes, día, horas, minutos, segundos);
let f4 = new Date(año, mes, día); // hora 00:00:00
```

| Leer          | Devuelve               | Cambiar          |
|---------------|------------------------|------------------|
| getDate()     | día del mes (1-31)     | setDate(día)     |
| getDay()      | día de la semana (0-6) | no tiene         |
| getMonth()    | mes (0-11)             | setMonth(mes)    |
| getFullYear() | año completo           | setFullYear(año) |
| getHours()    | hora                   | setHours(h)      |
| getMinutes()  | minutos                | setMinutes(m)    |
| getSeconds()  | segundos               | setSeconds(s)    |
| getTime()     | ms desde 1/1/1970      | setTime(ms)      |

| Método               | Resultado de ejemplo            |
|----------------------|---------------------------------|
| toString()           | Mon Dec 26 2050 08:30:00 GMT... |
| toDateString()       | Mon Dec 26 2050                 |
| toLocaleDateString() | 26/12/2050                      |
| toLocaleString()     | 26/12/2050 9:30:00              |
| toLocaleTimeString() | 9:30:00                         |

**Formato dd/mm/yyyy a mano** (sumar 1 al mes):

```
` ${f.getDate()}/${f.getMonth() + 1}/${f.getFullYear()} `
```

**Días entre dos fechas** (restar da milisegundos):

```
let dias = (fin - hoy) / (24 * 60 * 60 * 1000);
```

**Año pasado:** `f.setFullYear(f.getFullYear() - 1);`

**Día de la semana en español:**

```
let dias = ["Domingo", "Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado"];
dias[f.getDay()];
```

### Cumpleaños que cae en domingo hasta 2100:

```
for (let a = 2026; a <= 2100; a++) {
  if (new Date(a, mes - 1, dia).getDay() == 0)
    document.write(`${dia}/${mes}/${a}<br>`);
}
```

### Edad desde "dd/mm/yyyy":

```
let d = fecha.split("/"); // [dd, mm, yyyy]
let nac = new Date(d[2], d[1] - 1, d[0]);
let hoy = new Date();
let edad = hoy.getFullYear() - nac.getFullYear();
if (hoy < new Date(hoy.getFullYear(), nac.getMonth(), nac.getDate())) edad--;
```

**Saludo según la hora:** usar `new Date().getHours()` y comparar con números.

## 4. Math y aleatorios

No se crea con `new`; se llama directamente (`Math.PI`, `Math.round(x)`).

| Método                                                      | Hace                              | Ejemplo                                     |
|-------------------------------------------------------------|-----------------------------------|---------------------------------------------|
| <code>Math.round(x)</code>                                  | entero más cercano                | $3.75 \rightarrow 4$ , $3.25 \rightarrow 3$ |
| <code>Math.ceil(x)</code>                                   | redondea hacia arriba             | $3.25 \rightarrow 4$                        |
| <code>Math.floor(x)</code>                                  | redondea hacia abajo              | $3.75 \rightarrow 3$                        |
| <code>Math.trunc(x)</code>                                  | quita decimales                   | $-3.75 \rightarrow -3$                      |
| <code>Math.abs(x)</code>                                    | valor absoluto                    | $-4 \rightarrow 4$                          |
| <code>Math.max(a,b,c)</code> / <code>Math.min(a,b,c)</code> | mayor / menor                     |                                             |
| <code>Math.pow(base,exp)</code>                             | potencia                          | $\text{pow}(2,3) \rightarrow 8$             |
| <code>Math.sqrt(x)</code> / <code>Math.cbrt(x)</code>       | raíz cuadrada / cúbica            |                                             |
| <code>Math.PI</code>                                        | 3.14159...                        |                                             |
| <code>Math.random()</code>                                  | entre 0 (incluido) y 1 (excluido) |                                             |

```
Math.floor(Math.random() * 5);           // 0 a 4
Math.floor(Math.random() * 5) + 6;       // 6 a 10
Math.floor(Math.random() * (max - min + 1)) + min; // min a max, ambos
incluidos
```

**Moneda:** `Math.floor(Math.random() * 2) == 0 ? "Cara" : "Cruz"`

**Color aleatorio:** ``rgb(${Math.floor(Math.random()*256)} ,  
${Math.floor(Math.random()*256)} , ${Math.floor(Math.random()*256)})``

## 5. String y Number

El primer índice es 0 y el último `length - 1`. Las comillas invertidas permiten plantillas:  
``Hoy es ${dia}``.

| Método String                                              | Hace                                               | Ejemplo                                                   |
|------------------------------------------------------------|----------------------------------------------------|-----------------------------------------------------------|
| <code>length</code>                                        | nº de caracteres<br>(propiedad, sin<br>paréntesis) | <code>"hola".length → 4</code>                            |
| <code>charAt(i)</code> o <code>texto[i]</code>             | carácter en la posición i                          | <code>"leon".charAt(1) → "e"</code>                       |
| <code>substring(ini, fin)</code>                           | desde ini hasta fin SIN<br>incluir fin             | <code>"manzana".substring(1,3) → "an"</code>              |
| <code>split(sep)</code>                                    | divide en array                                    | <code>"01/01/2012".split("/") → ["01","01","2012"]</code> |
| <code>toUpperCase()</code> /<br><code>toLowerCase()</code> | mayúsculas / minúsculas                            |                                                           |
| <code>trim()</code>                                        | quita espacios al inicio y<br>final                |                                                           |
| <code>indexOf(x)</code>                                    | primera posición (-1 si no<br>está)                | <code>"jueves".indexOf("e") → 2</code>                    |
| <code>lastIndexOf(x)</code>                                | última posición                                    | <code>"jueves".lastIndexOf("e") → 4</code>                |
| <code>startsWith(x)</code> /<br><code>endsWith(x)</code>   | true o false                                       | <code>"Lunes".startsWith("Lu") → true</code>              |
| <code>replace(a, b)</code>                                 | sustituye solo la primera<br>aparición             | <code>"Hola<br/>Mundo".replace("Mundo","Universo")</code> |

Secuencias de escape: `\n` salto de línea, `\t` tabulador, `\"` comillas dobles, `\'` comillas simples, `\\` barra invertida.

| Método<br>Number | Hace                         | Ejemplo                                                    |
|------------------|------------------------------|------------------------------------------------------------|
| toFixed(n)       | n decimales (devuelve texto) | (12.123).toFixed(2) → "12.12";<br>(12.89).toFixed() → "13" |
| isNaN(x)         | true si NO es número         | isNaN("Hola") → true ; isNaN("15") → false                 |
| parseInt(x)      | texto a entero               | parseInt("37hola") → 37 ; parseInt("hola37") → NaN         |
| parseFloat(x)    | texto a decimal              | parseFloat("3.14") → 3.14                                  |
| Number(x)        | conversión estricta          | Number("37hola") → NaN                                     |

**Nombre y apellidos:** inicial + primer apellido + inicial segundo (btudor) → `datos.split(" ")` y `substring(0,1)` ; tres primeras letras de cada parte → `substring(0,3)` .

**Invertir cadena:** recorrer con `for (let i = t.length - 1; i >= 0; i--) r += t[i];`

**Nº de cifras de un número:** `String(n).length` o `prompt (ya es texto) .length` .

## 6. Evitar NaN (validar números)

NaN (Not a Number) sale cuando una operación numérica recibe algo que no es un número: `Number("hola")` , `parseInt("abc")` , `"Hola" * 5` , `undefined + 1` . Cualquier operación con NaN da NaN ( `5 + NaN` es NaN) y `NaN == NaN` es false, así que solo se detecta con `isNaN()` .

**Pedir un número hasta que sea válido (el más probable en examen):**

```
let n;
do {
  n = parseInt(prompt("Introduce un número"));
} while (isNaN(n));
document.write(`Has escrito ${n}`);
```

**Número entero positivo (si no, volver a preguntar):**

```
let n;
do {
  n = Number(prompt("Introduce un número entero positivo"));
} while (isNaN(n) || n <= 0);
```

**Comprobar una vez y avisar con alert:**

```
let n = Number(prompt("Introduce un número"));
if (isNaN(n)) {
  alert("No es un número");
} else {
  alert(`El doble es ${n * 2}`);
}
```

### Versión a prueba de todo (vacío y Cancelar):

```
let texto;
do {
  texto = prompt("Introduce un número");
} while (texto === null || texto.trim() === "" || isNaN(Number(texto)));
let n = Number(texto);
```

### Qué devuelve cada conversión:

| Valor                  | Number(valor) | parseInt(valor) | isNaN(valor) |
|------------------------|---------------|-----------------|--------------|
| "37"                   | 37            | 37              | false        |
| "3.5"                  | 3.5           | 3               | false        |
| "37hola"               | NaN           | 37              | true         |
| "hola37"               | NaN           | NaN             | true         |
| "" (vacío)             | 0             | NaN             | false        |
| null (pulsar Cancelar) | 0             | NaN             | false        |
| undefined              | NaN           | NaN             | true         |

Cuidado: `Number("")` y `Number(null)` dan 0, no NaN, y `isNaN("")` es false; por eso se comprueba aparte el texto vacío y el Cancelar, o se añade `n <= 0` cuando el número debe ser positivo. `parseInt("37hola")` da 37 y `Number("37hola")` da NaN: para validar de verdad es más estricto `Number`.

Para mostrar decimales: `n.toFixed(2)` devuelve texto con 2 decimales.

## 7. Función flecha (sintaxis)

Sin la palabra `function`; la flecha `=>` va entre los paréntesis de los parámetros y la llave. Se guarda en una variable y hay que declararla ANTES de llamarla.

```
// Tradicional
function suma(a, b) {
  return a + b;
}

// Anónima
let suma = function (a, b) {
  return a + b;
};

// Flecha
let suma = (a, b) => {
  return a + b;
};

suma(2, 3);    // 5
```

### Variantes:

```
let saludar = () => { alert("Hola"); };           // sin parámetros
let cuadrado = (n) => { return n * n; };          // un parámetro
let triple = (n = 0) => { return n * 3; };         // parámetro por defecto
let tabla = (filas = 10, cols = 4, color = "black") => { /* ... */ }; // varios por defecto
let doble = (n) => n * 2;                          // abreviada: sin {} ni
return (una sola expresión)
```

**Llamar:** `saludar();` · `let r = cuadrado(4);` · `triple();` (usa el 0 por defecto) · `tabla(20, 10);` (el tercero toma el valor por defecto).

Sin `return` la función devuelve `undefined`. Un parámetro de más al llamar se ignora: `triple(2, 3)` es 6.

## 8. Objeto window

`window` es el objeto principal del navegador: se crea uno por cada ventana que se abre desde la página y de él cuelga `document`. Desde la ventana padre se controlan las ventanas hijas (ver apartado 1).

| Propiedad             | Qué devuelve                     |
|-----------------------|----------------------------------|
| <code>closed</code>   | true si la ventana está cerrada  |
| <code>document</code> | el objeto document de la ventana |

| Propiedad                | Qué devuelve                                                                |
|--------------------------|-----------------------------------------------------------------------------|
| opener                   | la ventana que abrió la actual (con window.open)                            |
| parent                   | el padre de una ventana en un iframe                                        |
| self                     | la ventana actual                                                           |
| name                     | nombre de la ventana                                                        |
| location                 | URL del fichero                                                             |
| frames / length          | array de marcos (iframes) / número de marcos                                |
| innerWidth / innerHeight | ancho / alto en píxeles de la zona donde se ve la página                    |
| outerWidth / outerHeight | ancho / alto de toda la ventana (con barras y bordes)                       |
| scrollTop / scrollLeft   | píxeles desde el borde superior / izquierdo del navegador al de la pantalla |
| scrollX / scrollY        | píxeles de scroll horizontal / vertical                                     |

| Método                          | Qué hace                                      |
|---------------------------------|-----------------------------------------------|
| alert()                         | ventana de alerta con botón aceptar           |
| confirm()                       | ventana con aceptar y cancelar (true / false) |
| prompt()                        | ventana para introducir datos                 |
| open() / close()                | abre / cierra una ventana                     |
| focus() / blur()                | pone / quita el foco de la ventana            |
| moveBy(x, y) / moveTo(x, y)     | mueve la ventana relativo / a una posición    |
| resizeBy(x, y) / resizeTo(x, y) | cambia el tamaño sumando / a un tamaño exacto |
| scrollBy(x, y) / scrollTo(x, y) | mueve el scroll relativo / a una posición     |
| setTimeout() / clearTimeout()   | ejecuta una vez tras un tiempo / lo anula     |
| setInterval() / clearInterval() | ejecuta cada cierto tiempo / lo detiene       |

## 9. Trampas típicas

- `prompt` siempre devuelve texto: convertir con `Number()` / `parseInt()` antes de sumar o comparar. `"2" + 3` es `"23"`.
- Los meses de `Date` empiezan en 0: sumar 1 al mostrar, restar 1 al crear desde un dato del usuario.
- `getDay()` es el día de la semana (0 = domingo); `getDate()` es el día del mes.
- `setDate()` solo recibe el día; para el año usar `setFullYear()`.
- Los métodos llevan paréntesis: `toUpperCase()`, no `toUpperCase`. `length` es propiedad y no lleva.
- `substring(ini, fin)` no incluye la posición `fin`.
- `replace` cambia solo la primera aparición.
- En `setTimeout` / `setInterval` se pasa el nombre de la función sin `()`; tiempo en milisegundos.
- Siempre `clearInterval` al terminar una cuenta atrás o al cerrar la ventana.
- Tras mover o redimensionar una ventana hija, `ventana.focus()` para verla encima.
- Para que varias funciones compartan la referencia de la ventana, declarar la variable fuera de las funciones.
- `Math.random()` nunca devuelve 1; el aleatorio entre min y max incluidos lleva `+ 1` en el producto.
- `switch` sin `break` sigue ejecutando los casos siguientes.
- `var` sobrevive al bloque; `let` y `const` no. `const` no se puede reasignar.
- `==` compara valor; `===` compara valor y tipo.
- `isNaN("15")` es false (el texto "15" se convierte); `isNaN("Hola" * 5)` es true.